

FBI 先根据连续两帧点云推断物体怎样运动，再从运动推断手上哪里发生了接触，最后把视觉、接触信息和机器人状态一起送进一个单步生成式策略，输出灵巧手的动作序列。

FBI: Learning Dexterous In-hand Manipulation with Dynamic Visuotactile Shortcut Policy

Yijin Chen^{1*}, Wenqiang Xu^{1*}, Zhenjun Yu¹, Tutian Tang¹, Yutong Li¹, Siqiong Yao¹ and Cewu Lu¹

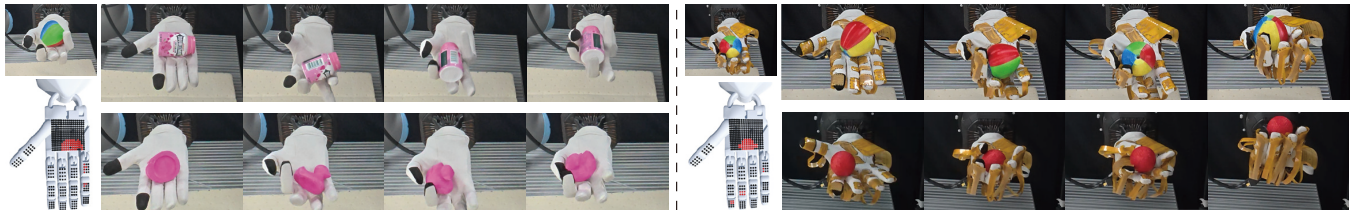


Fig. 1: We propose **Flow Before Imitation (FBI)**, a novel dynamic visuotactile imitation learning algorithm for dexterous in-hand manipulation. FBI’s design enables two operational modes: with or without physical tactile sensors in the real world, largely extending the application scenarios.

Abstract—Dexterous in-hand manipulation is a long-standing challenge in robotics due to complex contact dynamics and partial observability. While humans synergize vision and touch for such tasks, robotic approaches often prioritize one modality, therefore limiting adaptability. This paper introduces **Flow Before Imitation (FBI)**, a visuotactile imitation learning framework that dynamically fuses tactile interactions with visual observations through motion dynamics. Unlike prior static fusion methods, FBI establishes a causal link between tactile signals and object motion via a dynamics-aware latent model. FBI employs a transformer-based interaction module to fuse flow-derived tactile features with visual inputs, training a one-step diffusion policy for real-time execution. Extensive experiments demonstrate that the proposed method outperforms the baseline methods in both simulation and the real world on two customized in-hand manipulation tasks and three standard dexterous manipulation tasks. Code, models, and more results are available in the website <https://sites.google.com/view/dex-fbi>.

I. INTRODUCTION

In-hand manipulation, which aims to reposition objects within a single dexterous hand, remains a critical unsolved problem in robot learning due to complex contact dynamics and partial observability. Humans can easily interact with objects in hand thanks to the fusion of vision and touch. Vision tracks global object states for task completion, while tactile sensing enables precise force adjustments during contact. Existing robotic approaches often prioritize either vision [1]–[3] or tactile sensing [4]–[6], limiting their effectiveness in complex manipulation scenarios. While recent visuotactile approaches [5], [7]–[10] demonstrate improved manipulation stability, most rely on optical tactile sensors [5], [7]–[9] whose bulkiness (typically $\geq 8mm$ thickness) prevents deployment across full-hand articulations (e.g. metacarpal

joints). Distributed tactile arrays (e.g. piezoresistive arrays) present a practical alternative [4], [11], [12], offering hardware compatibility with commercial dexterous manipulators, full-palm coverage, and sufficient contact force resolution for detecting incipient slip. Despite these advantages, the fusion of distributed tactile data with visual streams remains understudied in policy learning frameworks.

Existing visuotactile fusion methods typically process multimodal inputs through *static fusion*, such as introducing encoded features [9] or augmenting visual point clouds with tactile contact coordinates [10] from a single frame. These approaches neglect the intrinsic causal relationship between tactile interactions and object state transitions during dynamic manipulation: Tactile forces drive object state changes. Therefore, we take the path of *dynamic fusion*, extracting tactile information from temporal object motion flow via a dynamics-aware latent model. This representation enables two operational modes in practice: (1) Vision-Only mode. We can infer tactile cues without physical sensors; (2) Visuotactile mode. With tactile sensors, measured contact forces can be used to refine flow predictions to improve precision. Therefore, our method can be flexibly deployed to a wider range of application scenarios, even when the tactile sensor is unavailable (Figure 1).

Building on this dynamic perspective, we present Flow Before Imitation (FBI) — an imitation learning framework integrating tactile and visual cues through motion dynamics. FBI processes multimodal inputs, including consecutive robot states, partially observed point clouds, and tactile readings, through dedicated encoder networks to extract proprioception and contact features. These encoded features are then dynamically fused and conditioned into a one-step shortcut model [13] to predict action sequences. This design enables efficient, real-time policy execution while integrating tactile-visual dynamics for robust in-hand manipulation.

To evaluate our method, we test the FBI on five tasks, including in-hand reorientation, in-hand pushing, and three dexterous tasks from a public benchmark, **Adroit** [14], with

¹Shanghai Jiao Tong University. * indicates equal contribution.
{st.czzz, vinjohn, jeffson-yu,
tttang, davidliyutong, yaosiqiong,
lucewu}@sjtu.edu.cn

objects ranging from the standard simple cubes to real-world items. Compared with four baseline methods including DP3 [15], ManiCM [16], Ada-Flow [17], and Consistency Policy [18], the proposed method performs better. In simulations, it reaches 64.7% (Vision-Only) to 66.5% (Visuotactile) average success, 16.6% to 18.4% higher than the previous SOTA method, respectively. It is especially good at hard reorientation tasks, 19.3% (Vision-Only) to 21.4% (Visuotactile) higher. We also conduct real-world tests that show similar gains, 33.5% (Vision-Only) to 35.0% (Visuotactile) vs. 18.5% baseline.

Our contribution can be summarized as follows:

- 1) We propose Flow Before Imitation (FBI), a visuotactile fusion method that dynamically infers tactile interactions from object motion flow, enabling real-time control via a one-step diffusion policy. It enables two operational modes with or without physical tactile sensors in the real-world experiments, largely extending the application scenarios.
- 2) Extensive evaluation across five tasks in both simulation and real-world experimental settings demonstrates the FBI's superiority over baseline methods.

II. RELATED WORKS

A. Multimodal Sensing for Dexterous Manipulation.

Considering that humans operate with dexterous manipulation, multimodal sensing, especially vision and touch, is of concern. To empower robots with such abilities, researchers often start with simplified, single-modal perception settings including vision-only [1]–[3], [19], [20] and touch-only [4]–[6]. Though much progress has been made, vision-only methods, by nature, struggle with occlusions, while tactile-only systems lack global spatial awareness. Recent advances highlight the benefits of multimodal sensing. Guzey *et al.* [8], [9] combined self-supervised tactile pre-training with visual inputs to address contact reasoning, outperforming single-modal baselines. Yuan *et al.* [10] introduced a point cloud-based tactile representation fused with vision to enhance spatial planning. While these methods demonstrate the potential of multimodal fusion, some works exclusively use finger contacts as effectors while neglecting palm interactions [5], [7], and others restrict object rotation to predefined axes [4], [10]. In contrast, our approach utilizes the full palm side to manipulate objects, enabling the ability to reorient irregular objects and execute controlled pushes.

B. Imitation Learning for Dexterous Manipulation.

Since analytical planning [21] or control [22] has limited generalization ability to object variance, learning for dexterous manipulation gains increasing attention. While reinforcement learning (RL) [3], [5], [7] excels without demonstrations, it requires intricate reward design. On the contrary, imitation learning methods can mitigate the reward design problem and are known to have better sample efficiency with demonstration data. The policy learning methods in imitation are developed along with the generative methods, from nearest neighbors-based approaches [8], [9], [19], Gaussian-based dynamic motion primitives [23], generative adversarial

networks [24], to diffusion models [15]–[18]. Ze *et al.* [15] replaced 2D image inputs in vanilla diffusion policy [25] with 3D point clouds, which significantly enhanced the performance of dexterous manipulation. Later, some works [17], [18], [26] have replaced the diffusion process with flow matching [13], [27] to reduce inference time. However, these methods only exploit a single modality.

III. METHOD

Dexterous in-hand manipulation requires rich hand-object interactions, where vision and touch complement each other. We propose Flow Before Imitation (FBI), a visuotactile imitation learning algorithm for this task. FBI takes two consecutive robot's state $s_{t-1}, s_t \in \mathbb{R}^{N_s}$, partially-observed point cloud frames $P_{t-1}, P_t \in \mathbb{R}^{N_p \times 3}$ and tactile readings $R_t \in \mathbb{R}^{N_r}$ as input, where N_s is the number of joints, N_p is the number of points in the downsampled point clouds (Section III-A), N_r is the number of contact keypoints (Section III-C). Such multimodal data are first processed by the multimodal encoders (Section III-A). After that, the encoded features are fused and passed as conditions to a shortcut model [13] to predict the action series $\mathbf{A}_t \in \mathbb{R}^{H \times d_a}$ (Section III-B), where H is the horizon of the action series, d_a is the dimension of each action.

While visuotactile systems excel in ideal conditions, we often witness hardware limitations in real-world deployments, since distributed tactile sensors could be unavailable or cost-prohibitive. To enhance accessibility and lower the barriers for labs and industries without tactile hardware, our method can maintain the dexterous manipulation capabilities using only visual input. Thanks to the dynamic perspective, tactile readings R_t can be predicted from the point cloud flow $f_{t-1 \rightarrow t} \in \mathbb{R}^{N_p \times 3}$ by the Flow2Tactile Module (Section III-C). An overview of our method is shown in Figure 2.

A. Multimodal Encoders

1) *Robot State Encoder*: We use a **3-layer MLP** as the state encoder $\mathbf{f}_s(\cdot)$ to encode the robot's two consecutive proprioception s_{t-1}, s_t into a compact state feature $\mathcal{F}_s \in \mathbb{R}^{d_s}$.

2) *Visual Encoder*: Given the two-frame point clouds of the hand and the object, we first crop and downsample them into N_p points and forward them to a **lightweight MLP** encoder $\mathbf{f}_v(\cdot)$. The lightweight encoder, consisting of a **three-layer MLP**, a **max-pooling function**, and an **output MLP layer for feature dimension reduction**, takes the downsampled point cloud P_{t-1} and P_t as input and outputs the compact feature $\mathcal{F}_v \in \mathbb{R}^{d_v}$, where d_v is the dimension of the visual feature. The visual feature will be fused with the tactile feature to form the visuotactile feature.

3) *Tactile Encoder*: To process the tactile readings, we use a **four-layer MLP** as the tactile encoder $\mathbf{f}_t(\cdot)$ to encode tactile feature $\mathcal{F}_{tac} \in \mathbb{R}^{d_{tac}}$, where d_{tac} is the dimension of the tactile feature. Note that in the vision-only setting, the tactile readings come from the prediction results of the Flow2Tactile module. In the visuotactile setting, where the physical tactile sensor is available, it comes as direct readings from the sensors.

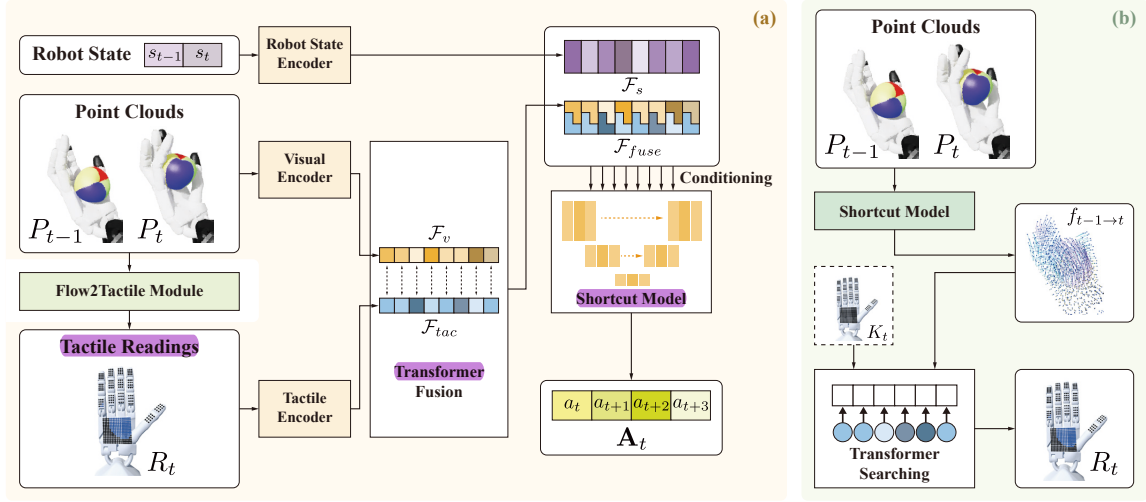


Fig. 2: **Overview of FBI pipeline.** (a) The overview of our pipeline, including **multimodal encoders**, **visuotactile feature fusing module**, and **the policy generation module**. (b) The **Flow2Tactile module** that **predicts the tactile readings, making it compatible with vision-only scenarios**.

B. Visuotactile Policy Generation

With the visual feature $\mathcal{F}_v$ and tactile feature $\mathcal{F}_{tac}$, we are able to obtain the visuotactile features and use them as conditions for the policy generation model. In this work, we adopt a shortcut model [13] to predict action series $\mathbf{A}_t$. The shortcut model is a flow-matching-based generative model that enables high-quality one-step sampling. In the training phase, starting from an initial inference step dt , it optimizes both flow matching loss and self-consistency loss simultaneously to improve the model's performance and increase the effective inference step size at the same time, ultimately achieving single-step generation ($dt = 1$). In the sampling phase, given condition C and a desired inference step size dt , the traditional flow-matching model can generate target distributions $p_\theta(x|C)$ from a standard normal distribution $x \sim \mathcal{N}(0, I)$. Starting from a Gaussian noise x^0 , it utilizes a U-net [28] as the velocity predicting network $v_\theta(\cdot)$ to recursively predicts the velocity at time step $t \in [0, 1]$ until finally reach the clear action x^1 at denoising time step 1:

$$x^{t+dt} = x^t + v_\theta(x^t, t, dt, C)dt. \quad (1)$$

In the shortcut model, which is one-step generation, we have $t = 0, dt = 1$.

1) **Fusing Tactile Features with Vision Features:** The encoded visual feature $\mathcal{F}_v$ and tactile feature $\mathcal{F}_{tac}$ are forwarded to a **transformer fusion module** $T_f(\cdot)$ which will take $\mathcal{F}_v$ as the **template feature**, $\mathcal{F}_{tac}$ as the **searching feature** and outputs the visuotactile feature. Considering visual observation is also crucial for environmental perception, we concatenate the visuotactile feature with the visual feature to form the **final fused feature** $\mathcal{F}_{fuse}$.

2) **Predicting Actions from Multimodal Conditions:** In our case, the model condition C consists of $\mathcal{F}_{fuse}$ and the robot state features $\mathcal{F}_s$. The velocity output by $v_\theta(\cdot)$ represents the denoising direction from the noisy action series $\mathbf{A}^0$ to the predicted action series $\mathbf{A}_t$. With the implementation of the

shortcut model, we can use the $v_\theta(\cdot)$ **only once** to complete the prediction:

$$\mathbf{A}_t = \mathbf{A}^t + v_\theta(\mathbf{A}^t, t, dt, \mathcal{F}_s, \mathcal{F}_{fuse})dt, \quad (2)$$

where $t = 0, dt = 1$ in our case. The t in $\mathbf{A}_t$ denotes the **time frame in the task progress**, while the t in $\mathbf{A}^t$ denotes the **denoising time step**. The benefits of the feature fusion on our policy will be included in Section IV-D.

3) **Training Objective:** We follow the main idea of training shortcut models to train our shortcut policy for one-step action generation. We first determine a minimum inference step size dt and optimize both **flow matching loss** and **self-consistency loss** simultaneously. Assume that $t \in [0, 1]$ is the randomly selected variable, $\mathbf{A}^0 \sim \mathcal{N}(0, I)$ is the starting action Gaussian noise, $\mathbf{A}^1$ is the ground-truth action series, $\mathcal{F} = \{\mathcal{F}_s, \mathcal{F}_{fuse}\}$ is the conditional feature during the training phase, $v_\theta(\cdot)$ is the velocity predicting model, then the loss function should be given as:

$$\begin{aligned} \mathcal{L}_{FM} &= \text{MSE}((\mathbf{A}^1 - \mathbf{A}^0), v_\theta(\mathbf{A}^t, t, dt, \mathcal{F})) \\ \mathcal{L}_{SC} &= \text{MSE}(v_\theta(\mathbf{A}^t, t, 2dt, \mathcal{F}), v_{target}) \\ \mathcal{L} &= \mathcal{L}_{FM} + \mathcal{L}_{SC}, \end{aligned} \quad (3)$$

如果一步跨越 $\sqrt{2}\Delta t$, 它应该与连续执行两个 Δt 小步近似一致

where $v_{target} = [v_\theta(\mathbf{A}^t, t, dt, \mathcal{F}) + v_\theta(\mathbf{A}^{t+dt}, t + dt, dt, \mathcal{F})] / 2$ and $\mathbf{A}^{t+dt} = \mathbf{A}^t + v_\theta(\mathbf{A}^t, t, dt, \mathcal{F})dt$.

C. Flow2Tactile Module

Dense contact information is crucial for contact-rich manipulation tasks, yet acquiring it demands high-precision tactile sensors, which are often unavailable in real-world experiments. Therefore, we leverage a **Flow2Tactile module** to predict dense contact states using object state flows. In doing so, the framework can also work in the vision-only mode.

Specifically, an **object state flow** $f_{t-1 \rightarrow t}$ is first predicted by a **shortcut model** using two frames point clouds P_{t-1} and P_t to represent how P_{t-1} transforms into P_t . Then, a pre-trained transformer searching model $T_s(\cdot)$ predicts

the tactile readings R_t on a pre-defined layout of contact keypoints utilizing $f_{t-1 \rightarrow t}$ and the current coordinate of contact keypoints $K_t \in \mathbb{R}^{N_k \times 3}$, obtaining the dense contact state at time step t . Here, N_k is the number of contact keypoints. The formation of the tactile processing module should be given as:

$$R_t = T_s(K_t, f_{t-1 \rightarrow t}). \quad (4)$$

The design of these contact keypoints aims to comprehensively cover the hand's palm side while referring to the physical parameters of sensors in the real world, so that real sensors can fit in the layout by finding a suitable correspondence between the real tactile sensor layout and the predefined contact keypoint layout. Take the Shadow Hand [29] as an example, a set of 456 points, consisting of 12 keypoints on each finger link and 288 keypoints on the palm, is used as our contact keypoints (Figure). We use **forward kinematics** to calculate **the current coordinates** $K_t \in \mathbb{R}^{456 \times 3}$ of the contact keypoints at time step t . We refer to some distributed tactile sensor designs [4], [11], [12], [30] and design keypoint layout for several hands. We display them in Figure 3.

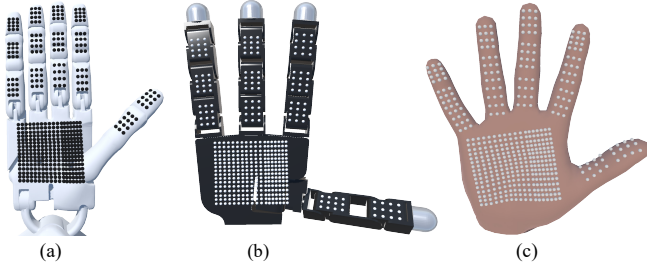


Fig. 3: Contact keypoint designs for (a): Shadow Hand [29], (b): Allegro Hand [31], (c): Mano Hand [11].

1) *Training Objectives*: The pre-trained Flow2Tactile module contains two parts: **the flow generation part** and **the transformer searching module**. We give the training losses directly:

$$\begin{aligned} \mathcal{L}_{FM} &= CD(P_t, (P_{t-1} + f_{t-1 \rightarrow t})) \\ \mathcal{L}_{SC} &= MSE(v_\theta(f^t, t, 2dt, P), v_{target}) \\ \mathcal{L} &= \mathcal{L}_{FM} + \mathcal{L}_{SC}, \end{aligned} \quad (5)$$

where **CD** stands for **Chamfer Distance**, v_{target} is alike Eq.3, only to substitute A^t to f^t , $\mathcal{F}$ to P .

For the transformer searching module, we compute the loss between the output and the ground-truth tactile readings R_{gt} , which is calculated in simulation (detailed in Section IV-B). The loss function is given as:

$$\mathcal{L}_t = MSE(R_{gt}, T_s(K_t, f_{t-1 \rightarrow t})). \quad (6)$$

IV. EXPERIMENTS

In this section, we systematically evaluate our method in both simulation and real experiments. Specifically, we evaluate the FBI in 2 in-hand-manipulation tasks on various objects in both simulation and real-world settings. We also select three dexterous manipulation tasks from a public benchmark for fair comparison with baseline methods.

A. Task Setup

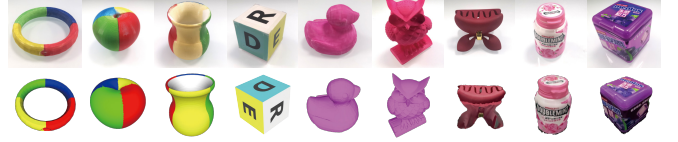


Fig. 4: Object dataset appearance in the real world (top row) and simulation (bottom row).

1) *Object Dataset*: An overview of our object dataset is shown in Figure 4. The object dataset contains objects of various shapes, weights, and colors. It has 6 3D-printed objects of both simple and complex shapes. The symmetric shapes are colored to indicate the orientation. We also have 3 daily objects to test the adaptability of our algorithm.

2) *Robot Hand*: For both simulation and real-world experiments, we utilized the Shadow Hand [29], a five-fingered robotic hand with 24 degrees of freedom.

3) *Task Description*: We evaluate FBI on two in-hand tasks: **Push** (moving objects to target positions) and **Reorientation** (adjusting orientations to target pose) in both simulation and real-world. Three Adroit tasks (**Door**, **Hammer**, **Pen**) [14] using Shadow Hand in MuJoCo [32] are also selected, following prior benchmarks [15], [16].

4) *Evaluation Metric and Goal Definition*: Success Rate (SR) is used for all tasks. Push success: object-target position $\leq 1\text{mm}$; Reorientation success: orientation difference $\leq 0.1\text{rad}$; Adroit tasks retain original criteria. SR is auto-calculated in simulation and manually assessed in the real world. In our own tasks, the target goal is a synthetic object point cloud transformed into the desired pose, which is concatenated with the observed scene point cloud and input to the model. For Adroit tasks, goals (door, hammer) are involved in the scene point cloud.

B. Training and Evaluation in Simulation

We simulate **In-hand Push** and **In-hand Reorientation** tasks using Isaac Sim 4.1.0 [33] (25 Hz control, 120 Hz simulation). **State-based PPO agents** [34], trained with learning rate $5e-4$ and training epochs $1e4$, achieving 65.7% success rate for in-hand reorientation and 96.8% for in-hand push, are used to collect 5000 success trajectories per object. For **Adroit** tasks, in MuJoCo, a **VRL3** [35] agent generates 10 success trajectories per task. FBI is trained for 300 epochs on the first two tasks and 3000 epochs on Adroit to ensure convergence.

To train the Flow2Tactile Module, we use ZeMa [36] to calculate the ground-truth tactile readings in simulation, because Isaac Sim doesn't currently support adding dense tactile sensors to robot hands and querying tactile readings. ZeMa [36] is a simulation platform based on the finite element method (FEM), featuring the most accurate contact modeling presently. It has been employed in some studies [12], [37], [38] to replicate real-world contact scenarios. Thus, we can treat the contact states calculated by ZeMa as ground-truth tactile readings. Specifically, we first collect the correlated states of hands and objects in the RL-generated

Algorithm \ Task	NFE	Cube	Apple	Vase	Ring	In-hand Duck	Reorientation Owl	daily object1	daily object2	daily object3	In-hand Push Balls	Hammer	Adroit Door	Pen	Average
FBI (Vision-Only)	1	80±3	86±2	74±5	76±1	28±5	27±4	65±4	39±2	25±7	95±3	100±0	77±3	69±4	64.7
FBI (Visuotactile)	1	84±4	90±2	80±2	76±8	28±2	29±3	68±2	42±5	22±10	97±1	100±0	75±2	73±3	66.5
DP3	10	51±9	53±12	44±2	50±5	20±5	18±3	49±2	24±5	18±6	84±7	100±0	69±5	45±5	48.1
ManiCM	1	50±3	50±10	46±3	53±2	17±2	15±6	47±6	25±7	16±4	86±5	100±0	68±3	43±6	47.4
Ada-Flow	1.82	15±12	30±2	21±4	15±7	1±1	3±2	30±7	25±2	10±3	44±15	50±12	52±3	28±4	24.9
Consistency Policy	1	19±8	32±3	20±3	18±6	1±2	2±0	25±10	26±3	12±3	48±10	55±6	50±7	26±5	25.7

TABLE I: **Simulation Results.** All baselines are run under the same simulation parameters for fair comparison. Lighter green indicates the second-best.

Algorithm \ Task	NFE	Cube	Apple	Vase	Ring	In-hand Duck	Reorientation Owl	daily object1	daily object2	daily object3	In-hand Push Balls	Average
FBI (Vision-Only)	1	45.0	50.0	35.0	40.0	20.0	15.0	30.0	25.0	15.0	60.0	33.5
FBI (Visuotactile)	1	45.0	50.0	40.0	40.0	15.0	20.0	30.0	25.0	20.0	65.0	35.0
DP3	10	10.0	15.0	5.0	5.0	0.0	0.0	10.0	5.0	0.0	20.0	7.0
ManiCM	1	25.0	25.0	30.0	15.0	10.0	5.0	20.0	10.0	15.0	30.0	18.5
Ada-Flow	1.32	10.0	10.0	10.0	15.0	0.0	0.0	15.0	15.0	5.0	25.0	10.5
Consistency Policy	1	15.0	10.0	15.0	15.0	5.0	0.0	10.0	15.0	10.0	20.0	11.5

TABLE II: **Real-World Experiment Results.** FBI outperforms all baselines in all tasks, achieving 15.0% (Vision-Only) to 16.5% (Visuotactile) improvement compared to the previous SOTA baseline, ManiCM. Lighter green indicates the second-best.

demonstrations. Then, we replay the trajectories in ZeMa, calculate the frame-wise tactile readings using incremental potential contact (IPC), and finally binarize them to obtain the ground-truth tactile readings. We choose 1000 trajectories per object and the average object density is 463.1 kg/m^3 . After training, our Flow2Tactile can reach an 85.5% prediction accuracy on unseen trajectories. Qualitative results of the ground-truth tactile readings (collected by ZeMa) and the predicted tactile readings (output by Flow2Tactile Module) in simulation is shown in Figure 5.

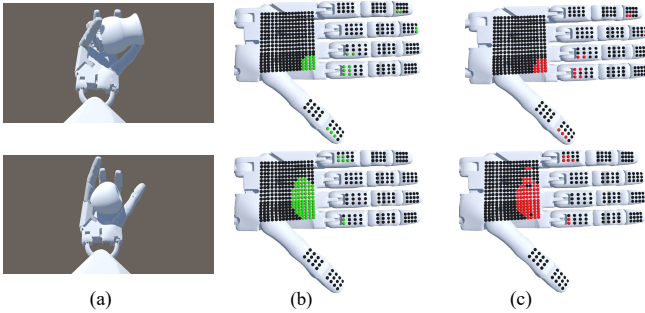


Fig. 5: Qualitative results of the Flow2Tactile Module. (a): The rendering result. (b): Ground-truth tactile readings (ZeMa). (c): Predicted tactile readings (Flow2Tactile Module).

C. Real-World Deployment

1) *Hardware Setup:* A Kinect Azure camera is used to capture visual observations. To mitigate metallic reflections degrading point clouds, we equip the Shadow Hand with a glove (Figure 6). For the visuotactile mode, Runeskee tactile sensors (0~5N range, 0.1N resolution, 20 Hz communication frequency) are bought from Taobao, molded on a flexible printed circuit (FPC), and attached to finger links and the palm. The layout of the sensor is customized to fit the robot hand, which is shown in Figure 7(a). It features 8 sensors on the first phalanx of each finger, 3 on the second phalanx

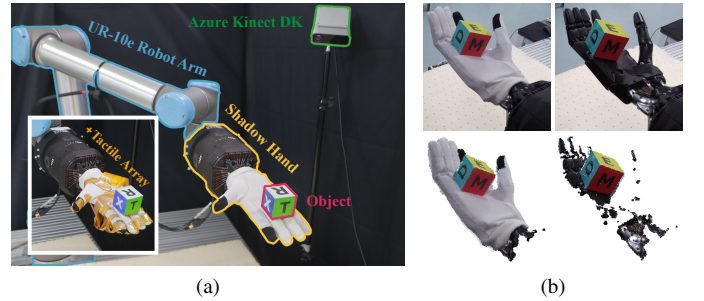


Fig. 6: (a): Real-world setup for both the vision-only mode and the visuotactile mode. (b): Images and point clouds were obtained by Kinect Azure before (right) and after (left) wearing gloves. The images are cropped versions that serve as input for image-based baselines.

except for the thumb, and 6 on the third phalanx. In total, there are 82 sensors on the fingers, 66 sensors on the palm, bringing a total of 148 sensors. To fit our real sensor into the contact keypoints, we establish a mapping from the tactile sensors to the contact keypoints. Specifically, we divide the hand into several square regions, using the lower-left corner of each square region as the origin to calculate the coordinates of both the tactile sensors and contact keypoints. Then, for each contact keypoint, we select the tactile sensors within the same region and apply bidirectional linear interpolation to the force readings on these tactile sensors to finally obtain the tactile readings on the contact keypoints. The regions are shown in Figure 7(b).

2) *Sim-to-Real Transfer:* The FBI can be directly employed in real-world experiments. On the visual side, thanks to the glove coverage, the real-world point cloud quality is sufficient to take the direct transfer (Figure 6). On the tactile side, with a correspondence between the real sensors and contact keypoints, we can easily convert sensor readings into contact keypoint readings, thanks to the binary tactile reading design. On the action side, to make the robot's behavior



Fig. 7: (a): The layout of the tactile sensors on the palm side (left) and the dorsal side (right) on the Shadow Hand in the real world. The tactile sensors are distributed on the palm side. (b): Regions for tactile readings mapping. Tactile sensors and contact keypoints are divided into 16 regions. Blocks of the same color indicate the corresponding regions.

smoother, we apply an exponential moving average to the predicted actions and an action interpolation between the target joint pose and the current joint pose. After that, the FBI can successfully perform dexterous manipulation tasks in the real world. The target joint states are generated at about 20 Hz on average, accounting for both the time of point cloud transmission and the model inference time.

D. Experiment Results

1) *Baselines*: We compare against four baselines: DP3 (point cloud diffusion) [15], ManiCM (point cloud consistency model) [16], Ada-Flow (image-based flow matching) [17], and Consistency Policy (image-based diffusion consistency) [18], selected to emphasize visuotactile feature fusion benefits. For image-based methods, inputs are cropped to hand-object regions with uniform resolution.

2) *Simulation Results*: For simulation tasks, three seeds (0,1,2) are tested. Each seed evaluates 20 episodes every 20 epochs, with top-5 success rates averaged per seed. We report the mean and standard deviation of success rates across three seeds. The simulation results presented in Table I reveal that, when averaged over all tasks, FBI achieves 16.6% (Vision-Only) to 18.4% (Visuotactile) improvement compared to DP3, with a smaller variance. In the In-hand Reorientation task, FBI surpasses DP3 with a 19.3% (Vision-Only) to 21.4% (Visuotactile) improvement averaged over 9 different objects, indicating that after fusing dense contact states with visual information, our algorithm demonstrates superior performance, particularly in handling more complex dexterous tasks. With the tactile information, FBI can lower the chance of objects getting stuck during the process, reach the target more smoothly, and reduce the occurrence of objects slipping from the hand.

3) *Real-World Experiment Results*: For real-world tasks, we evaluate each task with 20 trials and count the success rates. We keep hardware settings the same for every task to ensure fair comparison. Results in Table II indicate that our results in the simulation experiments remain similar in real-world settings. Moreover, it also shows that our algorithm, after sim-to-real processing, can be successfully applied to real-world scenarios. Similarly, with the implementation of tactile information and the shortcut model, the manipulation process in the real world becomes faster, smoother, and more

reliable. Qualitative results shown in Figure 8 illustrate two successful manipulation processes implemented in the real world.

E. Ablation Study

In the ablation study, we analyze data scale impact, Flow2Tactile importance, tactile input variations, fusion method variations, generalization, and shortcut model inference speed effects.

1) *Data Scale vs. Success Rate*: To validate our data scale for contact-rich tasks, we analyze success rate vs. trajectory count using in-hand reorientation as a case study (Table III). Considering the task complexity, it does not show promising results after 100 data samples. The influence of the data sample number converges after 5000 trajectories.

	Number of Successful Trajectories								
	10	100	500	1000	3000	4000	5000	6000	8000
SR	0.0	4.3	16.0	27.6	45.1	51.8	55.6	56.2	56.6

TABLE III: Relationship between SR and the number of successful trajectories. We test FBI (Vision-Only) on the In-hand Reorientation task in simulation. SR is averaged across 9 objects.

2) *Importance of Flow Prediction in Tactile Information Generation*: We compare contact state generation methods to evaluate the tactile processing module: 1) Flow2Tactile (Ours): the contact states are generated by searching the pre-placed keypoints with the predicted flow; 2) Point Cloud (PC) to Tactile: the contact states are generated by searching the pre-placed keypoints with two-frame point clouds directly; 3) TOCH [39]: a hand-object contact modeling method based on spatio-temporal correspondences, yet suffers from heavy optimization and slow real-time inference 4) Ground Truth: We use the ground-truth tactile readings R_{gt} (mentioned in Section III-C) calculated in simulation. Results are shown in Table IV.

Algorithm \ Task	In-hand Reorientation	In-hand Push	Adroit	Average
Flow2Tactile (Ours)	55.6	95.0	82.0	64.7
PC to Tactile	47.6	90.0	77.0	57.6
TOCH	53.9	94.0	83.3	63.8
Ground Truth	58.3	97.0	84.0	67.2

TABLE IV: SR results on different tactile generation methods. We include the ground-truth tactile readings R_{gt} as a reference.

3) *Different Forms of Tactile Information*: To demonstrate the benefits of the **dense contact** information, we conduct experiments on different forms of tactile information: 1) Dense Binary Contact (Ours): The contact states consist of binary tactile readings on 456 contact keypoints; 2) Dense Continuous Contact: The contact states consist of continuous tactile readings (force applied to the tactile sensors) on 456 contact keypoints; 3) Sparse Binary Contact: The contact states consist of link-wise binary tactile readings (readings between objects and the 24 links on the Shadow Hand);

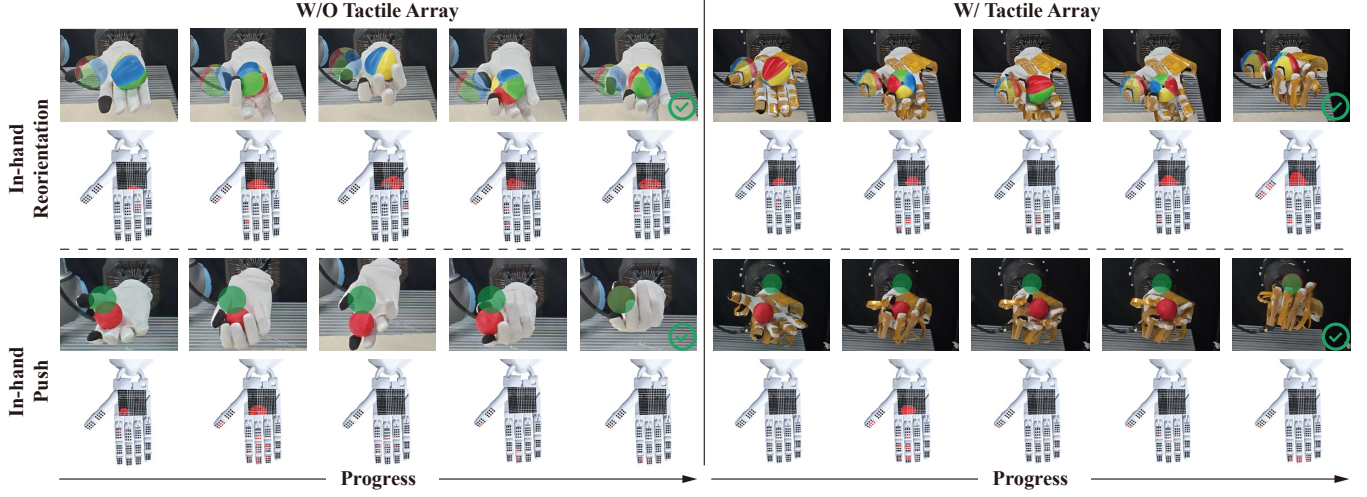


Fig. 8: **Qualitative results of the In-hand Reorientation task and the In-hand Push task.** We demonstrate both the vision-only mode (left) and the visuotactile mode (right). We capture and display five key frames from each manipulation process and the corresponding tactile readings.

4) W/o Contact: The policy model works without contact conditions. Results are shown in Table V.

Algorithm \ Task	In-hand Reorientation	In-hand Push	Adroit	Average
Dense Binary (Ours)	55.6	95.0	82.0	64.7
Dense Continuous Contact	56.7	93.0	83.3	65.6
Sparse Contact	38.2	87.0	73.3	50.1
W/o Contact	36.4	87.0	70.3	48.1

TABLE V: Success Rate results on different tactile information. Lighter green indicates the second-best.

From Table V, we can observe that dense tactile information, either binary or continuous, greatly enhances the model’s performance in dexterous tasks. In contrast, joint-wise sparse tactile information only offers slight gains. Although dense continuous contact provides the best performance, we choose dense binary contact as our representation because it simplifies sim-to-real transfer and improves the accuracy of model predictions, as binary predictions are significantly easier for neural networks compared to regression.

4) *Benefits of the Transformer-based visuotactile fusing approach:* To demonstrate the effectiveness of the visuotactile fusing module, we conduct experiments on different fusing methods: 1) Transformer Fusion (Ours): The visual and tactile features are forwarded to our transformer fusion module; 2) MLP Fusion: A multi-layer MLP is used to fuse the visuotactile features; 3) Add: We add the visual and tactile features to obtain the fused feature. Results are shown in Table VI.

Algorithm \ Task	In-hand Reorientation	In-hand Push	Adroit	Average
Transformer Fusion (Ours)	55.6	95.0	82.0	64.7
MLP Fusion	48.1	92.0	77.3	58.2
Add	43.2	89.0	75.7	44.3

TABLE VI: Success Rate results on different fusing method. Lighter green indicates the second-best.

5) *Generalization Ability to Unseen Objects:* FBI demonstrates promising generalization to unseen object sizes/shapes. In cube reorientation, both FBI and DP3 succeed on original objects, but FBI outperforms DP3 on

novel variants in zero-shot tests (Table VII), attributed to the contact modeling. We use FBI (Vision-Only) for fair comparison.

Unseen Shape	Algorithms	Size					Chamfered				
		NO.1	NO.2	NO.3	NO.4	NO.5	NO.1	NO.2	NO.3	NO.4	NO.5
0.9 × Size	FBI (VO)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DP3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
0.8 × Size	FBI (VO)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DP3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
0.75 × Size	FBI (VO)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DP3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
1.1 × Size	FBI (VO)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DP3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Chamfered Cube (Param.0.8)	FBI (VO)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DP3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Chamfered Cube (Param.0.6)	FBI (VO)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	DP3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

TABLE VII: **Size and shape generalization on cubes.** Each unseen size/shape is evaluated with 5 different initial and target poses, denoted No.1-No.5. VO stands for Vision-Only.

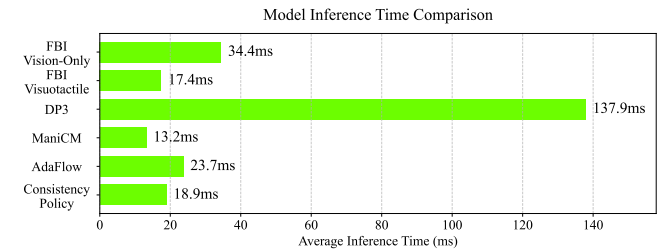


Fig. 9: Average model inference time of all baselines in real-world experiments. We record the latency from receiving inputs (images/ point clouds/ robot states) to the final action series for all baselines for fairness.

6) *Details on The Model's Inference Speed:* We now detail the inference speed in the real world and justify the use of shortcut models [13]. We report the **average model inference time** (denoted as AVG Time) for all baselines on an NVIDIA RTX 4090 GPU in real-world experiments in Figure 9. FBI (Vision-Only) achieves 20 Hz control (51.2ms per action series), including environment interaction (16.8ms) and model inference (34.4ms). it spends 17.4ms on the Flow2Tactile Module (flow prediction: 11.2ms, tactile reading prediction: 6.2ms) and 15.0ms on the visuotactile policy (Transformer fusion: 4.8ms, shortcut policy: 10.2ms), with 2.0ms overhead. FBI (Visuotactile) reduces the model inference time to 17.4ms (15.2ms for the policy and 2.2ms overhead) by skipping the Flow2Tactile Module. In contrast, DP3 [15] requires 137.9ms for model inference, and even its simplified variant (Simple DP3) spends 83.6ms. Replacing FBI's shortcut models with 10-step DDIM [40] would inflate latency to 261.4ms, underscoring the necessity of our design.

V. CONCLUSION

This work introduces Flow Before Imitation (FBI), a visuotactile fusion framework that advances in-hand manipulation by dynamically linking tactile interactions to object motion dynamics. FBI's dynamics-aware latent model enables synergy between vision and touch, supporting robust performance with or without physical tactile sensors. Evaluations across 5 dexterous tasks in both simulated and real-world settings demonstrate FBI's superiority, achieving a 18.4% higher success rate than state-of-the-art baselines, with notable gains (21.4%) in in-hand reorientation. Critically, the FBI maintains stability under partial sensor failure, enhancing deployability in practical settings.

REFERENCES

- [1] K. Xu, Z. Hu, R. Doshi, A. Rovinsky, V. Kumar, A. Gupta, and S. Levine, "Dexterous manipulation from images: Autonomous real-world rl via substep guidance," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5938–5945.
- [2] Y. Qin, Y.-H. Wu, S. Liu, H. Jiang, R. Yang, Y. Fu, and X. Wang, "Dexmv: Imitation learning for dexterous manipulation from human videos," in *European Conference on Computer Vision*. Springer, 2022, pp. 570–587.
- [3] O. M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray *et al.*, "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.
- [4] Z.-H. Yin, B. Huang, Y. Qin, Q. Chen, and X. Wang, "Rotating without seeing: Towards in-hand dexterity through touch," *arXiv preprint arXiv:2303.10880*, 2023.
- [5] M. Yang, C. Lu, A. Church, Y. Lin, C. Ford, H. Li, E. Psomopoulou, D. A. Barton, and N. F. Lepora, "Anyrotate: Gravity-invariant in-hand object rotation with sim-to-real touch," *arXiv preprint arXiv:2405.07391*, 2024.
- [6] K.-W. Lee, Y. Qin, X. Wang, and S.-C. Lim, "Dextouch: Learning to seek and manipulate objects with tactile dexterity," *IEEE Robotics and Automation Letters*, 2024.
- [7] H. Qi, B. Yi, S. Suresh, M. Lambeta, Y. Ma, R. Calandra, and J. Malik, "General in-hand object rotation with vision and touch," in *Conference on Robot Learning*. PMLR, 2023, pp. 2549–2564.
- [8] I. Guzey, B. Evans, S. Chintala, and L. Pinto, "Dexterity from touch: Self-supervised pre-training of tactile representations with robotic play," *arXiv preprint arXiv:2303.12076*, 2023.
- [9] I. Guzey, Y. Dai, B. Evans, S. Chintala, and L. Pinto, "See to touch: Learning tactile dexterity through visual incentives," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 13 825–13 832.
- [10] Y. Yuan, H. Che, Y. Qin, B. Huang, Z.-H. Yin, K.-W. Lee, Y. Wu, S.-C. Lim, and X. Wang, "Robot synesthesia: In-hand manipulation with visuotactile sensing," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6558–6565.
- [11] S. Sundaram, P. Kellnhofer, Y. Li, J.-Y. Zhu, A. Torralba, and W. Matusik, "Learning the signatures of the human grasp using a scalable tactile glove," *Nature*, vol. 569, no. 7758, pp. 698–702, 2019.
- [12] C. Jiang, W. Xu, Y. Li, Z. Yu, L. Wang, X. Hu, Z. Xie, Q. Liu, B. Yang, X. Wang *et al.*, "Capturing forceful interaction with deformable objects using a deep learning-powered stretchable tactile array," *Nature Communications*, vol. 15, no. 1, p. 9513, 2024.
- [13] K. Frans, D. Hafner, S. Levine, and P. Abbeel, "One step diffusion via shortcut models," *arXiv preprint arXiv:2410.12557*, 2024.
- [14] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," *arXiv preprint arXiv:1709.10087*, 2017.
- [15] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy," *arXiv e-prints*, pp. arXiv–2403, 2024.
- [16] G. Lu, Z. Gao, T. Chen, W. Dai, Z. Wang, W. Ding, and Y. Tang, "Manicm: Real-time 3d diffusion policy via consistency model for robotic manipulation," *arXiv preprint arXiv:2406.01586*, 2024.
- [17] X. Hu, Q. Liu, X. Liu, and B. Liu, "Adaflow: Imitation learning with variance-adaptive flow-based policies," *Advances in Neural Information Processing Systems*, vol. 37, pp. 138 836–138 858, 2024.
- [18] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, "Consistency policy: Accelerated visuomotor policies via consistency distillation," *arXiv preprint arXiv:2405.07503*, 2024.
- [19] S. P. Arunachalam, S. Silwal, B. Evans, and L. Pinto, "Dexterous imitation made easy: A learning-based framework for efficient dexterous manipulation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5954–5961.
- [20] A. Handa, A. Allshire, V. Makovychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviychuk, K. Van Wyk, A. Zhurkevich, B. Sundaralingam *et al.*, "Dextreme: Transfer of agile in-hand manipulation from simulation to reality," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5977–5984.
- [21] S. Cruciani, C. Smith, D. Kragic, and K. Hang, "Dexterous manipulation graphs," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 2040–2047.
- [22] F. Khadivar and A. Billard, "Adaptive fingers coordination for robust grasp and in-hand manipulation under disturbances and unknown dynamics," *IEEE Transactions on Robotics*, vol. 39, no. 5, pp. 3350–3367, 2023.
- [23] A. Hammoud, V. Belcamino, A. Carfi, V. Perdureau, and F. Mastrogiorgio, "In-hand manipulation planning using human motion dictionary," in *2022 31st IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*. IEEE, 2022, pp. 927–933.
- [24] D. Antotsiou, G. Garcia-Hernando, and T.-K. Kim, "Task-oriented hand motion retargeting for dexterous manipulation imitation," in *Proceedings of the European conference on computer vision (ECCV) workshops*, 2018, pp. 0–0.
- [25] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, p. 02783649241273668, 2023.
- [26] Y. Su, X. Zhan, H. Fang, Y.-L. Li, C. Lu, and L. Yang, "Motion before action: Diffusing object motion as manipulation condition," *arXiv preprint arXiv:2411.09658*, 2024.
- [27] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," *arXiv preprint arXiv:2210.02747*, 2022.
- [28] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–241.
- [29] ShadowRobot, "Shadow hand," <https://www.shadowrobot.com/>.
- [30] H. Liu, X. Xie, M. Millar, M. Edmonds, F. Gao, Y. Zhu, V. J. Santos, B. Rothrock, and S.-C. Zhu, "A glove-based system for studying

- hand-object manipulation via joint pose and force sensing,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 6617–6624.
- [31] WonikRobotics, “Allegrohand,” <https://www.wonikrobotics.com/>, 2013.
 - [32] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.
 - [33] NVIDIA, “Isaac sim - robotics simulation and synthetic data generation,” <https://developer.nvidia.com/isaac-sim>, 2023, (accessed on May 2, 2023).
 - [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
 - [35] C. Wang, X. Luo, K. Ross, and D. Li, “Vrl3: A data-driven framework for visual deep reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 32 974–32 988, 2022.
 - [36] W. Du, S. Yao, X. Wang, Y. Xu, W. Xu, and C. Lu, “Intersection-free robot manipulation with soft-rigid coupled incremental potential contact,” *IEEE Robotics and Automation Letters*, 2024.
 - [37] S. Ma, W. Du, C. Yu, Y. Jiang, Z. Zong, T. Xie, Y. Chen, Y. Yang, X. Han, and C. Jiang, “Grip: A general robotic incremental potential contact simulation dataset for unified deformable-rigid coupled grasping,” *arXiv preprint arXiv:2503.05020*, 2025.
 - [38] Z. Yu, W. Xu, P. Xie, Y. Li, and C. Lu, “Dynamic reconstruction of hand-object interaction with distributed force-aware contact representation,” *arXiv preprint arXiv:2411.09572*, 2024.
 - [39] K. Zhou, B. L. Bhatnagar, J. E. Lenssen, and G. Pons-Moll, “Toch: Spatio-temporal object-to-hand correspondence for motion refinement,” in *European Conference on Computer Vision*. Springer, 2022, pp. 1–19.
 - [40] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” *arXiv preprint arXiv:2010.02502*, 2020.

1. 什么叫“接触中间表示”？

假设我们有连续点云：

$$P_{t-1}, P_t$$

以及机器人关节状态 s_t 。

最直接的端到端方法是：

$$a_t = \pi_\theta(P_{t-1}, P_t, s_t).$$

也就是说，网络直接从点云预测动作。中间所有东西——物体运动、接触位置、是否滑动——都藏在神经网络内部的 latent feature（潜在特征）里。

FBI 则显式地把问题分解：

$$(P_{t-1}, P_t) \xrightarrow{g_\phi} f_{t-1 \rightarrow t} \xrightarrow{h_\psi} \hat{R}_t \xrightarrow{\pi_\theta} A_t.$$

分别表示：

1. $f_{t-1 \rightarrow t}$ ：物体点云运动流；
2. $\hat{R}_t$ ：手表面的稠密二值接触图；
3. A_t ：未来一段关节动作。

其中

$$\hat{R}_t \in \{0, 1\}^{456},$$

第 i 个元素表示 Shadow Hand 上第 i 个预定义位置是否接‘’ 体。